Detailed Study Notes: Object-Oriented Programming in Java (Packages, Static, Singleton Class & In-built Methods)

1. Overview and Structural Scope

In Object-Oriented Programming (OOP) with Java, managing code structure, memory allocation, and class-level execution contexts is fundamental to constructing robust applications. As presented in the lecture series by Kunal Kushwaha, these detailed study notes provide a structured, topic-by-topic breakdown of four essential concepts in Java programming:

1. **Packages and Code Organization** : Modular grouping of classes, resolving naming conflicts, and directory path mappings.
2. **The static Keyword** : Class-level variables, static methods, static initialization blocks, and static nested classes.
3. **The Singleton Design Pattern** : Restricting class instantiation to a single shared object instance across the application lifetime.
4. **In-Built Java Methods and System Internals** : Architectural dissection of System.out.println() and overriding the toString() method.

2. Topic 1: Packages and Code Organization in Java

2.1 Definition and Purpose of Packages

In Java, a **package** is a container or directory that groups related classes, interfaces, and sub-packages together. As explained in the lecture, packages address a primary architectural challenge in software development: **naming collisions** .
When building enterprise applications or integrating external libraries, developers often need to create classes with identical names (such as Main, Greeting, or Message). Without packages, Java would not allow two classes with the same name within the same project. Packages establish distinct compartments or namespaces, allowing identically named classes to exist without conflict in separate folders.

2.2 Directory Hierarchy and Naming Conventions

Java enforces a strict relationship between package declarations and the underlying folder structure on disk starting from the project's src root. Standard industry convention dictates using a company's reverse domain name to guarantee global package uniqueness:

```
// File location: src/com/kunal/packages/a/Greeting.java
package com.kunal.packages.a;

public class Greeting {
    public static void main(String[] args) {
        System.out.println("Hello from Package A");
    }
}
```

If another Greeting class is required in the project, it is placed in a separate package directory:

```
// File location: src/com/kunal/packages/b/Greeting.java
```

```
package com.kunal.packages.b;

public class Greeting {
    public static void main(String[] args) {
        System.out.println("Hello from Package B");
    }
}
```

2.3 Package Declarations and Import Mechanics

The package statement must be the very **first line of code** in a Java file. It declares the exact logical location of the class within the project hierarchy.
To utilize a public class located in another package, Java requires an explicit import statement specifying the path to the target class:

```
package com.kunal.packages.b;

// Importing a specific class from Package A
import com.kunal.packages.a.Message;

public class Main {
    public static void main(String[] args) {
        // Invoking a method from Package A's Message class
        Message.showMessage();
    }
}
```

Java automatically imports the java.lang.* package into every Java file, providing built-in access to classes like String, System, Math, and Object. Utility packages, such as java.util.Scanner, must be explicitly imported.

2.4 Access Control Across Package Boundaries

As highlighted in the lecture, access modifiers control whether classes and members in one package are accessible to code in another package. The table below compares cross-package visibility rules for basic access modifiers:
Access Modifier,Visible Within Same Package?,Visible Outside Package?
public,Yes,Yes (Requires explicit import statement)
Default (Package-Private),Yes,No (Triggers a compilation error)
Declaring a member public makes it accessible across package boundaries, whereas omitting an access modifier restricts access strictly to classes within the same package.

3. Topic 2: The static Keyword — Class-Level Variables and Execution Contexts

3.1 Concept of Static

The static keyword in Java is used to declare variables, methods, blocks, and nested classes that belong to the **class itself** rather than to individual **instances (objects)** of the class.
When a member is declared as static:

- It is loaded into memory when the class is first loaded by the Java Virtual Machine (JVM).
- It exists independently of any object creation.
- It is shared globally among all instances of that class.

3.2 Static Variables: Shared Class Attributes

Instance variables (such as name, age, and salary) hold distinct values for each object. Conversely, a static variable represents a property shared across all instances of a class. The lecture demonstrates this using a Human class where population is a shared count incremented upon every object creation:

```
public class Human {
    // Instance variables (unique to each object instance)
    String name;
    int age;
    int salary;
    boolean married;

    // Static variable (shared globally across all Human objects)
    static long population;

    public Human(String name, int age, int salary, boolean
married) {
        this.name = name;
        this.age = age;
        this.salary = salary;
        this.married = married;

        // Increment class-level population variable inside
constructor
        Human.population += 1;
    }
}
```

Access Convention for Static Variables

While Java technically permits accessing a static variable through an object reference (e.g., kunal.population), doing so generates compiler warnings. The standard convention is to access static variables directly using the **Class Name** :

```
public class Main {
    public static void main(String[] args) {
        Human kunal = new Human("Kunal", 22, 10000, false);
        Human rahul = new Human("Rahul", 25, 15000, true);

        // Recommended Access Pattern: Via Class Name
        System.out.println(Human.population); // Output: 2
    }
}
```

3.3 Static Methods

A static method can be executed without creating an instance of the class containing it.

Dissecting public static void main(String args)

The primary entry point of any Java program is the main method. As detailed in the source material, main **must** be declared static because:

1. main is the first routine executed by the JVM when a program starts.
2. At program execution, no instances of the Main class exist in memory.
3. If main were non-static, the JVM would face a logical deadlock: it would need to instantiate Main to run main, but instantiation logic requires executing code first. Declaring main static enables the JVM to call Main.main() directly without prior object instantiation.

3.4 Interaction Rules Between Static and Non-Static Contexts

Java enforces strict rules governing how static and non-static methods and data interact. The table below outlines these accessibility rules:

Caller Context,Target Member Type,Direct Access Allowed?,Mechanics / Requirement
Static Context,Static Member,Yes,Both belong to the class level.
Static Context,Non-Static Member,No (Compilation Error),Static context lacks an object reference (this). Must instantiate an object first.
Non-Static Context,Static Member,Yes,Non-static context retains class-level visibility.
Non-Static Context,Non-Static Member,Yes,Resolves against the current instance (this).

Calling Non-Static Routines from Static Contexts

To execute a non-static method from within a static method (such as main), an object instance must be explicitly instantiated inside the static context:

```
public class Main {
    public static void main(String[] args) {
        // Direct call 'greeting()' causes a compilation error:
        // "Non-static method greeting() cannot be referenced from
a static context"

        // Correct approach: Instantiate an object explicitly
        Main obj = new Main();
        obj.greeting();
    }

    // Non-static method belonging to instance objects of Main
    public void greeting() {
        System.out.println("Hello World");
    }
}
```

Absence of this and super Keywords in Static Methods

In Java, the this keyword represents a reference to the **current object instance** . Because static methods exist independently of any instance, this (and super) cannot be referenced inside static methods or static blocks.

4. Topic 3: Static Initialization Blocks

4.1 Purpose and Execution Order

Static variables often require complex calculation or multi-step initialization logic. Java provides **static initialization blocks** (static { ... }) to handle class-level initialization.
Key properties of static blocks demonstrated in the lecture:

- A static block runs **exactly once** when the class is first loaded into JVM memory.
- It executes prior to any constructor invocations or object creations.
- Instantiating subsequent objects does **not** re-execute the static block.

4.2 Code Demonstration of Static Initialization Blocks

The following code example shows how static variables are initialized inside a static block and proves its single-execution lifecycle:

```
public class StaticBlockDemo {
    static int a = 4;
    static int b;

    // Static Initialization Block (runs once upon class loading)
    static {
        System.out.println("Executing Static Initialization
Block");
        b = a * 5; // b initialized to 20
    }

    public static void main(String[] args) {
        System.out.println("Main method started");
        System.out.println("a = " + StaticBlockDemo.a + ", b = " +
StaticBlockDemo.b);

        // Modifying static variable b
        StaticBlockDemo.b += 3; // b becomes 23

        // Creating first instance of StaticBlockDemo
        StaticBlockDemo obj1 = new StaticBlockDemo();
        System.out.println("obj1 b = " + StaticBlockDemo.b); // 23

        // Creating second instance of StaticBlockDemo
        StaticBlockDemo obj2 = new StaticBlockDemo();
        System.out.println("obj2 b = " + StaticBlockDemo.b); // 23
(Static block DOES NOT run again)
    }
}
```

Output:

```
Executing Static Initialization Block
Main method started
a = 4, b = 20
obj1 b = 23
obj2 b = 23
```

5. Topic 4: Nested / Inner Classes and Static Qualification

5.1 Top-Level vs. Nested Classes

Top-level (outermost) classes in Java **cannot be marked as static** . Outer classes can only be public or package-private.
However, **nested (inner) classes** —classes defined inside an outer class—can be qualified as static.

5.2 Mechanics of Static Inner Classes

Declaring an inner class as static means that the inner class does **not require an instance of the outer enclosing class** to be instantiated.
As analyzed in the lecture:

- A non-static inner class depends on an instance of the outer class (Outer outer = new Outer(); Outer.Inner inner = outer.new Inner();).
- A static inner class can be instantiated directly (Test a = new Test("Kunal");).

```
public class MainOuter {

    // Static Nested Class
    static class Test {
        String name;

        public Test(String name) {
            this.name = name;
        }
    }

    public static void main(String[] args) {
        // Instantiate static inner class directly within static
main
        Test a = new Test("Kunal");
        Test b = new Test("Rahul");

        // Each instance maintains its own distinct 'name' field
        System.out.println(a.name); // Output: Kunal
        System.out.println(b.name); // Output: Rahul
    }
}
```

Even though Test is a static class, its instance variable name remains unique to each object (a.name is "Kunal", b.name is "Rahul"). The static modifier on the class declaration only uncouples Test from needing an instance of MainOuter.

6. Topic 5: The Singleton Design Pattern

6.1 Concept and Application

The **Singleton Design Pattern** is a creational pattern designed to restrict the instantiation of a class to **exactly one object instance** across the application lifecycle.
Singletons are used when a centralized system resource is needed, such as database connection managers, logging engines, or configuration handlers.

6.2 Step-by-Step Implementation Pattern

To build a Singleton class, three specific implementation steps are followed:

1. **Private Constructor** : Prevents external classes from creating instances via new Singleton().
2. **Private Static Reference Variable** : Holds the single created instance (private static Singleton instance).
3. **Public Static Factory Method** : Implements a global access method (getInstance()) with lazy initialization.

```
public class Singleton {
    // 1. Private static variable holding the single instance
reference
    private static Singleton instance;

    // 2. Private constructor blocks direct external instantiation
    private Singleton() {
        System.out.println("Singleton Object Instantiated");
    }

    // 3. Public static getter providing controlled global access
    public static Singleton getInstance() {
        // Lazy Initialization: Instantiate object only on first
request
        if (instance == null) {
            instance = new Singleton();
        }
        return instance; // Return existing single instance
reference
    }
}
```

6.3 Memory Reference Verification

As demonstrated in the lecture, when multiple reference variables invoke getInstance(), all references point to the exact same heap memory location:

```
public class Main {
    public static void main(String[] args) {
```

```
        // Requesting instance references
        Singleton obj1 = Singleton.getInstance();
        Singleton obj2 = Singleton.getInstance();
        Singleton obj3 = Singleton.getInstance();

        // Memory Reference Identity Checks
        System.out.println(obj1 == obj2); // Output: true
        System.out.println(obj2 == obj3); // Output: true
    }
}
```

7. Topic 6: In-Built Java Methods and System Internals

7.1 Structural Breakdown of System.out.println()

While System.out.println() is standard syntax in Java, analyzing its composition clarifies core object-oriented principles:

- **System** : An in-built final class in the java.lang package.
- **out** : A public static final reference variable declared inside System of type java.io.PrintStream.
- **println()** : An instance method defined inside the PrintStream class that outputs data to the console.

Because out is declared static within System, it can be accessed directly as System.out without instantiating the System class.

7.2 Object Representation and Overriding toString()

When an object reference is passed to System.out.println(obj), Java internally calls String.valueOf(obj), which invokes obj.toString().
By default, every Java class inherits from java.lang.Object. The default toString() implementation returns a composite string of the class name and the object's hashcode in hexadecimal format (ClassName@hashCode).

Overriding toString() for Readable Output

To print meaningful field values when printing an object, developers override the default toString() method:

```
public class Student {
    String name;
    int rollNo;

    public Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }

    // Overriding toString() inherited from Object class
    @Override
    public String toString() {
```

```
        return "Student{name='" + name + "', rollNo=" + rollNo +
"}";
    }

    public static void main(String[] args) {
        Student s1 = new Student("Kunal", 101);

        // Internally calls s1.toString()
        System.out.println(s1);
        // Output: Student{name='Kunal', rollNo=101}
    }
}
```

The table below contrasts default versus overridden toString() behavior:
Implementation Type,Method Origin,Example Output,Primary Purpose
Default toString(),Inherited from java.lang.Object,Student@15db9742,Memory reference identity string.
Overridden toString(),Defined in Custom Class,"Student{name='Kunal', rollNo=101}","Clean, human-readable state summary."

8. Summary Knowledge Checklist

Review the core concepts below to confirm mastery of the topics covered in these notes:

1. **Packages** : Modularize classes into directory namespaces to eliminate class name collisions.
2. **Static Ownership** : Static members belong to the class object, existing independently of instance creation.
3. **Static Limitations** : Static routines cannot access non-static data directly and cannot reference this or super.
4. **Static Initialization Blocks** : Execute exactly once when the class is loaded by the JVM.
5. **Singleton Design Pattern** : Enforces a single instance using a private constructor and a static getter with null-checking lazy initialization.
6. **In-Built Mechanics** : System.out.println() utilizes the static PrintStream object out, and overriding toString() customizes object printing output.